ResNet 코드 구현 분석

Deep Residual Learning for Image Recognition

He et al., CVPR 2016

논문의 Residual Block 개념이
코드에서 어떻게 표현되는가

오늘 다룰 내용

- 이론 파트에서 들은 **Residual Block**, **Shortcut Connection**이 어떻게 구현되는지
- BasicBlock VS BottleNeck 차이
- plain34 VS ResNet34 : 코드 실험
- ResNet34 VS ResNet50 : 코드 실험

파일 구조 한눈에

```
resnet_18 코드
├── class BasicBlock   ← 재료 (반복되는 부품)
└── class ResNet18     ← 모델 (재료를 쌓아 만든 완성품)
```

ResNet-50 파일 구조 한눈에

```
resnet_50 코드
├── class Bottleneck   ← 재료 (1×1 → 3×3 → 1×1 병목 구조)
└── class ResNet50     ← 모델 (Bottleneck을 16개 쌓아 만듦)
```

각 클래스의 내부 구조

class BasicBlock

__init__

conv1, bn1, relu, conv2, bn2, shortcut을 `self.xxx`에 저장 (레이어 정의).

forward(x)

conv→bn→relu→conv→bn을 거친 결과에 shortcut으로 우회한 `x`를 더함 (`out += identity`).

class ResNet18

__init__

stem conv1, layer1~4, avgpool, fc를 `self.xxx`에 저장.

_make_layer(...)

BasicBlock을 지정 개수(`blocks`)만큼 반복해서 쌓아 `nn.Sequential`로 묶음.

forward(x)

이미지 `x`를 stem → layer1~4 → avgpool → fc 순으로 흘려 클래스 점수 반환.

ResNet-50의 내부 구조

class Bottleneck

__init__

conv1(1×1), bn1, conv2(3×3), bn2, conv3(1×1), bn3, shortcut 저장. `expansion=4`로 출력 채널 4배 확장.

forward(x)

1×1(축소) → 3×3 → 1×1(확장) 후 shortcut의 `x`를 더함 (`out += identity`).

class ResNet50

__init__

stem conv1, layer1~4, avgpool, fc(2048→N) 저장. 블록 (3,4,6,3) — ResNet-34와 같은 수, 종류만 Bottleneck.

_make_layer(...)

Bottleneck을 `blocks`개 반복해서 쌓음. 2번째부터 입력 채널은 `out_channels × 4`.

forward(x)

stem → layer1~4 → avgpool → fc 흐름 (ResNet18과 동일).

BasicBlock vs Bottleneck 비교

BasicBlock (ResNet-18/34)

```
# Conv 2개 — 같은 채널 유지
self.conv1 = nn.Conv2d(in_ch, out_ch,
    3, stride, 1, bias=False)   # 3×3
self.bn1   = nn.BatchNorm2d(out_ch)

self.conv2 = nn.Conv2d(out_ch, out_ch,
    3, 1, 1, bias=False)        # 3×3
self.bn2   = nn.BatchNorm2d(out_ch)
```

Bottleneck (ResNet-50/101/152)

```
# Conv 3개 — 1×1→3×3→1×1 병목 구조
self.conv1 = nn.Conv2d(in_ch, out_ch,
    1, bias=False)              # 1×1 축소
self.bn1   = nn.BatchNorm2d(out_ch)

self.conv2 = nn.Conv2d(out_ch, out_ch,
    3, stride, 1, bias=False)   # 3×3
self.bn2   = nn.BatchNorm2d(out_ch)

self.conv3 = nn.Conv2d(out_ch, out_ch*4,
    1, bias=False)              # 1×1 확장
self.bn3   = nn.BatchNorm2d(out_ch*4)
```

핵심 차이점

항목	BasicBlock	Bottleneck
Conv 개수	**2개** (3×3, 3×3)	**3개** (1×1, 3×3, 1×1)

Bottleneck 채널 흐름

```
                          out_channels      out_channels × 4
                          (병목 중간)       (외부 출력)
                             ↓                 ↓
입력 64 ──[1×1]──▶ 64 ──[3×3]──▶ 64 ──[1×1]──▶ 256
      in_channels        64는 그대로              ↑
                                              최종 출력
```

- `out_channels = 64` → 병목의 **좁은 가운데**
- `out_channels × 4 = 256` → 이 블록이 내보내는 **최종 출력 채널**
- `in_channels`은 이전 블록의 출력. 첫 블록은 stem(64)에서 받음

잔차연결 구현

논문에서

- 입력 `x`에 몇 층을 거친 결과 `F(x)`를 더함
- 출력 = `F(x) + x`
- "잔차(residual)"만 학습하면 되는 구조

코드에서

- `F(x)` = Conv→BN→ReLU→Conv→BN
- `+ x` = shortcut 경로로 입력을 그대로 더함
- `out += identity` 한 줄이 논문의 핵심

```python
def forward(self, x):
    identity = self.shortcut(x)
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)
    out = self.conv2(out)
    out = self.bn2(out)
    out += identity          # ← 잔차 연결 (F(x) + x)
    out = self.relu(out)
    return out
```

BasicBlock 블록도

```
입력 x
  │
  ├──────────────────────┐
  │                      │   identity = shortcut(x)
  ▼                      │
conv1 (3×3)              │
bn1                      │
relu                     │
conv2 (3×3)              │
bn2                      │
  │                      │
  + ◄────────────────────┘   F(x) + x
  │
relu
  ▼
```

Class ResNet50 구조

class ResNet50

__init__

stem conv1, layer1~4, avgpool, fc(2048→N)를 `self.xxx`에 저장.

_make_layer(...)

Bottleneck을 지정 개수(`blocks`)만큼 반복해서 쌓아 `nn.Sequential`로 묶음.

forward(x)

이미지 `x`를 stem → layer1~4 → avgpool → fc 순으로 흘려 클래스 점수 반환.

ResNet50 `__init__`

```python
def __init__(self, num_classes=1000):
    super(ResNet50, self).__init__()

    self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    self.bn1   = nn.BatchNorm2d(64)
    self.relu  = nn.ReLU(inplace=True)

    self.layer1 = self._make_layer(64,   64,  blocks=3, stride=1)
    self.layer2 = self._make_layer(256,  128, blocks=4, stride=2)
    self.layer3 = self._make_layer(512,  256, blocks=6, stride=2)
    self.layer4 = self._make_layer(1024, 512, blocks=3, stride=2)

    self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
    self.fc      = nn.Linear(2048, num_classes)   # 512 × 4 = 2048
```

- `blocks=3, 4, 6, 3`: 각 layer에 쌓는 **Bottleneck 개수** (layer1=3, layer2=4, layer3=6, layer4=3 → 총 16개)
- ResNet-34도 같은 분포 (3,4,6,3)지만 **블록 종류**가 다름 (BasicBlock vs Bottleneck)

`_make_layer` 헬퍼 함수 (ResNet50)

```
def _make_layer(self, in_channels, out_channels, blocks, stride):
    layers = []
    layers.append(Bottleneck(in_channels, out_channels, stride))      # 첫 블록
    for _ in range(1, blocks):
        layers.append(Bottleneck(
            out_channels * Bottleneck.expansion,                      # 이전 출력 = out × 4
            out_channels, stride=1))
    return nn.Sequential(*layers)
```

- **왜 필요?** Bottleneck을 여러 개 쌓는 반복을 한 번에 처리
- **첫 블록**: `in_channels` 받아서 확장 처리 (stride 적용)
- **이후 블록**: 입력 채널이 `out_channels × 4` (앞 블록의 출력)
- Bottleneck은 **내부(out)** 와 **외부(out × 4)** 가 다르므로 이 구분이 필수

ResNet50 `forward`

```
def forward(self, x):
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)                # → 32×32×64

    x = self.layer1(x)              # → 32×32×256   (Bottleneck×3)
    x = self.layer2(x)              # → 16×16×512   (Bottleneck×4)
    x = self.layer3(x)              # →  8×8×1024   (Bottleneck×6)
    x = self.layer4(x)              # →  4×4×2048   (Bottleneck×3)

    x = self.avgpool(x)             # →  1×1×2048
    x = x.view(x.size(0), -1)       # →  2048
    x = self.fc(x)                  # →  10 (클래스)
    return x
```

`layer1→layer4`로 갈수록 **해상도↓, 채널↑** — ResNet-18과 같은 흐름, 단 채널 수가 **4배** 큼.

블록도의 의미 — 텐서 흐름

블록	코드	출력 shape
Input	원본 이미지 (RGB)	32×32×3
Conv 3×3, 64	`self.conv1(x)`	32×32×**64**
Layer1 (Bottleneck×3)	`self.layer1(x)`	32×32×**256**
Layer2 (Bottleneck×4)	`self.layer2(x)`	**16×16**×512
Layer3 (Bottleneck×6)	`self.layer3(x)`	**8×8**×1024
Layer4 (Bottleneck×3)	`self.layer4(x)`	**4×4**×2048
AvgPool	`self.avgpool(x)`	**1×1**×2048
FC → 10	`self.fc(x)`	**10**

실험 결과: ResNet-34 vs Plain-34

CIFAR-10, 30 epoch, 동일 조건 (batch 128, SGD+Cosine, lr 0.1)

	ResNet-34	Plain-34	차이
정확도	90.72%	84.61%	**+6.11%p**
에러율	**9.28%**	**15.39%**	**−6.11%p** (에러 40% 감소)
학습 시간	41.6분	39.1분	+2.5분 (+6%)

핵심 발견

- 에러율이 **40% 감소**. 딥러닝에서 매우 큰 차이
- 시간 오버헤드는 겨우 **6%**

논문 재현

- Shortcut 하나 추가한 것만으로 ResNet이 Plain을 압도

실험 결과: ResNet-34 vs ResNet-50

	ResNet-34 (BasicBlock)	ResNet-50 (Bottleneck)
정확도	**90.72%**	82.07%
에러율	**9.28%**	17.93%
학습 시간	41.6분	**93.9분** (2.3배)

예상 밖 결과

- ResNet-50이 **8.65%p 낮음**, 2배 오래 걸림
- CIFAR-10(32×32)이 ResNet-50(ImageNet용)에 너무 작음

교훈

- ImageNet: ResNet-50 > ResNet-34
- CIFAR-10: ResNet-34 > ResNet-50 ← 우리 실험
- **모델은 데이터 규모에 맞춰야 함**